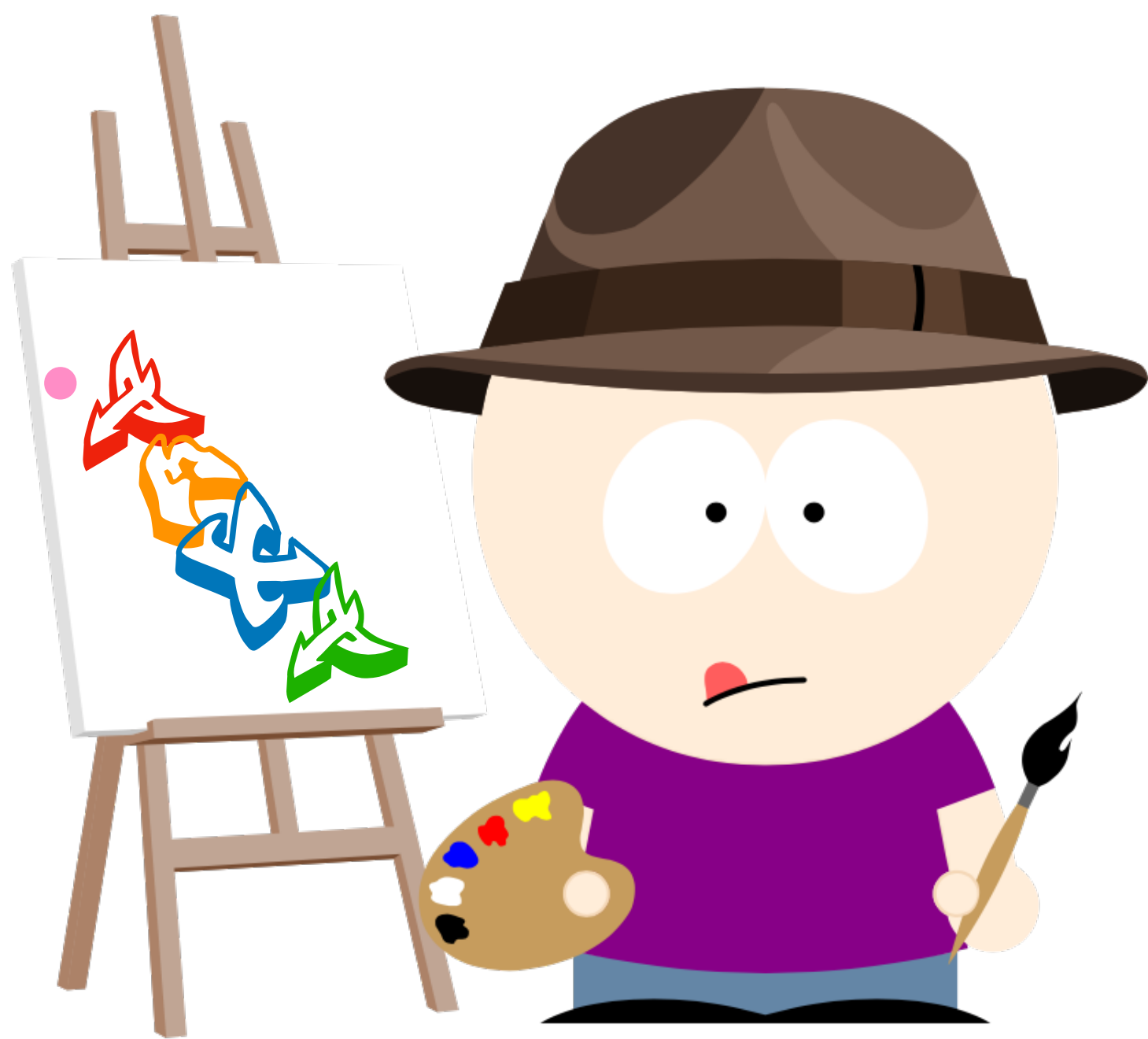




# IN-CLASS EXERCISES



# CODE GENERATION 1

# Download exercises

1. Go to

<https://ligerlabs.org/compilers.html>

2. Download the file

`codegen-1-exercises.zip`

3. Open up a terminal and Unzip the file

```
> unzip codegen-1-exercises.zip
```

```
> cd codegen-1-exercises
```

```
> ls
```

# Task 1

- Define these terms:
  1. Instruction selection
  2. Instruction scheduling
  3. Register allocation
  4. Register assignment
  5. Register spilling



# Task 2

- Translate (by hand!) this function to MIPS assembly code:

```
int foo () {  
    int x = 101;  
    int y = 11;  
    int z;  
    z = x ^ y;  
    z = z * 23;  
    return z;  
}
```

# Task 3

- Consider the MIPS function on the next 3 slides.
- Explain what statements 1-4 do.
- Can you translate these statements back to C code?

```
# Setting up the function
```

```
addiu    $sp, $sp, -16
```

```
sw       $ra, 12($sp)
```

```
sw       $fp, 8($sp)
```

```
move     $fp, $sp
```

```
# Statement 1
```

```
addiu    $1, $zero, 42
```

```
sw       $1, 4($fp)      # variable x
```

```
# Statement 2
```

```
lw       $1, 4($fp)
```

```
sll      $1, $1, 2
```

```
sw       $1, 0($fp)      # variable z
```



# Statement 3

```
lw      $1, 0($fp)
addiu   $2, $zero, 0
negu    $1, $1
sw      $1, 0($fp)
```

# Statement 4

```
lw      $1, 0($fp)
addiu   $1, $1, 1
sw      $1, 0($fp)
```

```
# Returning from the function
lw      $2, 0($fp)
move     $sp, $fp
lw      $fp, 8($sp)
lw      $ra, 12($sp)
addiu    $sp, $sp, 16
jr       $ra
```



# Task 4

- Consider this C program.
- The corresponding MIPS assembly code is on the next page and in the file `task4.s`.

1. Run the program using the SPIM emulator:

```
> spim -file task4.s
```

2. Optimize the code by keeping variable `i` in a dedicated register throughout the code. Test the code under SPIM again to make sure it's correct.

```
int i=1;
main () {
    for(i=1; i<=10; i++) {
        printf(i);
    }
}
```

```

        .data
_i:      .word 0
_nl:     .asciiz "\n"
        .text
        .globl main

main:
    li    $14, 1
    sw    $14, _i

$32:
    lw    $a0, _i

```

```

li    $v0, 1
syscall

li    $v0, 4
la    $a0, _nl
syscall

lw    $9, _i
addu  $10, $9, 1
sw    $10, _i
ble   $10, 10, $32

li    $v0, 10
syscall

```





# DEFINITIONS AND ALGORITHMS



# 32-bit Registers

| Register | Number    | Acronym Explanation                     |
|----------|-----------|-----------------------------------------|
| zero     | 0         | Constant 0                              |
| at       | 1         | Reserved for assembler                  |
| v0       | 2         | Expression evaluation and               |
| v1       | 3         | results of a function                   |
| a0       | \$4-\$7   | Argument 1-4                            |
| t0       | \$8-\$15  | Temporary (not preserved across call)   |
| s0       | \$16-\$23 | Saved temporary (preserved across call) |
| t8       | \$24-\$25 | Temporary (not preserved across call)   |
| k0       | \$26-\$27 | Reserved for OS kernel                  |
| gp       | 28        | Pointer to global area                  |
| sp       | 29        | Stack pointer                           |
| fp       | 30        | Frame pointer                           |
| ra       | 31        | Return address (used by function call)  |

# Addressing Modes

| Format                      | Address Computation                                            |
|-----------------------------|----------------------------------------------------------------|
| register                    | Contents of register.                                          |
| immediate                   | 16-bit constant value.                                         |
| imm (register)              | Contents of register +<br>16-bit constant value.               |
| symbol $\pm$ imm (register) | Address of symbol + or -<br>(immediate + contents of register) |

# Arithmetic Instructions 1

| Instruction                         | Semantics                                                                                             |
|-------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>abs Rdest, Rsrc</code>        | Put the absolute value of the integer from Rsrc in Rdest                                              |
| <code>add Rdest, Rsrc1, Src2</code> | Addition (with overflow). Same for sub.                                                               |
| <code>addi Rdest, Rsrc1, Imm</code> | Put the sum of the integers from register Rsrc1 and (an immediate) into register Rdest. Same for sub. |



# Arithmetic Instructions 2

| Instruction                   | Semantics                                                                                                        |
|-------------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>move Rdest, Rsrc</code> | Move the contents of Rsrc to Rdest.                                                                              |
| <code>neg Rdest, Rsrc</code>  | Put the negative of the integer from register Rsrc into register Rdest.                                          |
| <code>div Rsrc1, Rsrc2</code> | Divide the contents of the two registers.<br>Leave the quotient in register lo and the remainder in register hi. |

# Arithmetic Instructions 3

| Instruction                          | Semantics                                                                                                                               |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>mult Rsrc1, Rsrc2</code>       | Multiply the contents of the two registers.<br>Leave the low-order word of the product in register lo and the high-word in register hi. |
| <code>div Rdest, Rsrc1, Src2</code>  | Put the quotient of the integers from register Rsrc1 and Src2 into register Rdest.                                                      |
| <code>mulo Rdest, Rsrc1, Src2</code> | Put the product of the integers from register Rsrc1 and Src2 into register Rdest.                                                       |



# Logical Operations 1

| Instruction                         | Semantics                                                                                                       |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>and Rdest, Rsrc1, Src2</code> | Put the logical AND of the integers from register Rsrc1 and Src2 into register Rdest.<br>Same for or, xor, nor. |
| <code>andi Rdest, Rsrc1, Imm</code> | Put the logical AND of the integers from register Rsrc1 and Imm into register Rdest.<br>Same for or, xor, nor.  |
| <code>not Rdest, Rsrc</code>        | Put the bitwise logical negation of the integer from register Rsrc into register Rdest.                         |

# Shifts & Rotations 1

| Instruction                         | Semantics                                                                                                                                                                                               |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>rol Rdest, Rsrc1, Src2</code> | Rotate Left. Rotate the contents of register Rsrc1 left by the distance indicated by Src2 and put the result in register Rdest. Same for ror, (Rotate Right).                                           |
| <code>sll Rdest, Rsrc1, Src2</code> | Shift Left Logical. Shift the contents of register Rsrc1 left by the distance indicated by Src2 and put the result in register Rdest. Same for sra (Shift Right Arithmetic), srl (Shift Right Logical). |

# Literal Integer Loads 1

| Instruction                 | Semantics                                                                                                                                                                             |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>li Rdest, imm</code>  | Move the immediate <code>imm</code> into register <code>Rdest</code> .                                                                                                                |
| <code>lui Rdest, imm</code> | Load Upper Immediate. Load the lower halfword of the immediate <code>imm</code> into the upper halfword of register <code>Rdest</code> . The lower bits of the register are set to 0. |



# Comparison Instructions 1

| Instruction                         | Semantics                                                                                                                                                                                                                                                                                                   |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>seq Rdest, Rsrc1, Src2</code> | <p>Set Equal. Set register Rdest to 1 if register Rsrc1 equals Src2 and to be 0 otherwise. Same for sge, sgt, sle, slt, sne.</p> <p>Branch instructions use a signed 16-bit offset field; hence they can jump <math>2^{15}-1</math> instructions forward or <math>2^{15}</math> instructions backwards.</p> |
| <code>b label</code>                | <p>Branch instruction. Unconditionally branch to the instruction at the label. The jump instruction contains a 26 bit address field.</p>                                                                                                                                                                    |

# Comparison Instructions 2

| Instruction                         | Semantics                                                                                                                                              |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>beq Rsrc1, Src2, label</code> | Branch on Equal. Conditionally branch to the instruction at the label if the contents of register Rsrc1 equals Src2. Same for bge, bgt, ble, blt, bne. |
| <code>beqz Rsrc, label</code>       | Branch on Equal Zero. Conditionally branch to the instruction at the label if the contents of Rsrc equals 0. Same for bgez, bgtz, blez, bltz, bnez.    |

# Jump Instructions

| Instruction            | Semantics                                                                                                                              |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>j label</code>   | Jump Unconditionally. Jump to the instruction at the label.                                                                            |
| <code>jal label</code> | Jump and Link register. Unconditionally jump to the instruction at the label. Save the address of the next instruction in register 31. |
| <code>jalr Rsrc</code> | Jump and Link register. Unconditionally jump to the instruction whose address is in register Rsrc.                                     |
| <code>jr Rsrc</code>   | Jump Register. Unconditionally jump to the instruction whose address is in register Rsrc.                                              |



# Load & Store Instructions

| Instruction                    | Semantics                                                                                                                                                                                                                         |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>la Rdest, address</code> | Load Address. Load computed address, not the contents of the location, into register Rdest.                                                                                                                                       |
| <code>lb Rdest, address</code> | Load Byte. Load the byte at address into register Rdest. The byte is sign-extended. Same for <code>ld</code> (Load Double-Word [64 bits]), <code>lh</code> (Load Half-Word [16 bits]), and <code>lw</code> (Load Word [32 bits]). |
| <code>sb Rsrc, address</code>  | Store the low byte from register Rsrc at address. Same for <code>sd</code> (64 bits), <code>sh</code> (16 bits), and <code>sw</code> (32 bits).                                                                                   |

# Assembler Directives

| Syntax                          | Description                                                                       |
|---------------------------------|-----------------------------------------------------------------------------------|
| <code>.data</code>              | The following data items should be stored in the data segment.                    |
| <code>.float f1, ..., fn</code> | Store the floating point single precision numbers in successive memory locations. |
| <code>.space n</code>           | Allocate n bytes of space in the data segment.                                    |
| <code>.text</code>              | The next items are put in the user text segment.                                  |
| <code>.ascii "string"</code>    | Store the string in memory and null-terminate it.                                 |
| <code>.align n</code>           | Align the next datum on a $2^n$ byte boundary.                                    |
| <code>.word w1, ..., wn</code>  | Store the n 32-bit quantities in successive memory words.                         |
| <code>.byte b1, ..., bn</code>  | Store the n byte values in successive bytes of memory.                            |

# Floating Point Coprocessor

- The MIPS has single precision (32-bit) and double precision (64-bit) floating point (FP) numbers.
- Instructions operate on FP registers  $\$f0 - \$f31$ .
- Two 32-bit registers are required to hold doubles.
- FP operations only use even-numbered registers.
- Values are moved in or out of FP registers a word (32-bits) at a time using instructions `lwc1`, `swc1`, `mtc1`, and `mfc1`.



# Floating Point Instructions 1

| Instruction                               | Semantics                                                                                                                                                                                                           |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>abs.s FRdest, FRsrc</code>          | Floating Point Absolute Value Single. Compute the absolute value of the floating float double (single) in register FRsrc and put it in register FRdest.                                                             |
| <code>neg.s FRdest, FRsrc</code>          | Negate Single. Negate the floating point single in register FRsrc and put it in register FRdest.                                                                                                                    |
| <code>add.s FRdest, FRsrc1, FRsrc2</code> | Floating Point Addition Single. Compute the sum of the floating float singles in registers FRsrc1 and FRsrc2 and put it in register FRdest. Same for <code>div.s</code> , <code>mul.s</code> , <code>sub.s</code> . |

# Floating Point Instructions 2

| Instruction                        | Semantics                                                                                                                                                                               |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>c.eq.s FRsrc1, FRsrc2</code> | Compare Equal Single. Compare the floating point single in register FRsrc1 against the one in FRsrc2 and set the floating point condition flag true if they are equal. Same for c.le.s. |
| <code>cvt.s.w FRdest, FRsrc</code> | Convert Integer to Single. Convert the integer in register FRsrc to a single precision number and put it in register FRdest.                                                            |
| <code>cvt.w.s FRdest, FRsrc</code> | Convert Single to Integer. Convert the single precision floating point number in register FRsrc to an integer and put it in register FRdest.                                            |

# Floating Point Instructions 3

| Instruction                      | Semantics                                                                                          |
|----------------------------------|----------------------------------------------------------------------------------------------------|
| <code>mov.s FRdest, FRsrc</code> | Move Floating Point Single. Move the floating float single from register FRsrc to register FRdest. |
| <code>l.s FRdest, address</code> | Load Floating Point Single. Load the floating float single at address into register FRdest.        |
| <code>s.s FRdest, address</code> | Store Floating Point Single. Store the floating point single in register FRdest at address.        |



**COLLBERG.CS.ARIZONA.EDU**

**LIGERLABS.ORG**

**SUPPORTED BY**  
**NSF SATC/TTP-1525820**  
**SATC/EDU-2029632**

**COPYRIGHT © 2024-2026**

**CHRISTIAN COLLBERG**

**UNIVERSITY OF ARIZONA**